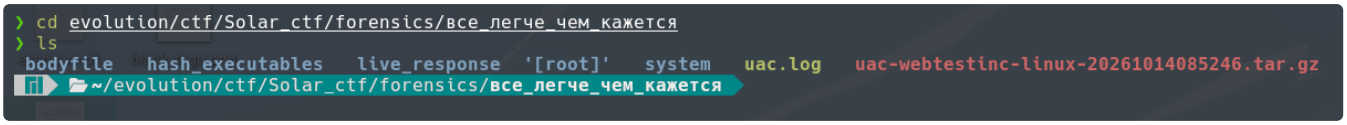
Всё_легче_чем кажется

Описание:

> При реагировании вы наткнулись на веб-сервер для тестов веб-сайтов. Атакующие успели побывать и на нём. Ваша задача — найти вредонос и достать из него флаг. Система не перезагружалась с момента атаки.



Итак, нам дали архив `UAC` это сборщик артефактов Linux. Но самое интересное в условии `сервер не перезагружался`. Значит, начнём не с логов, а с процессов: возможно, вредонос ещё работал, когда снимали артефакты.

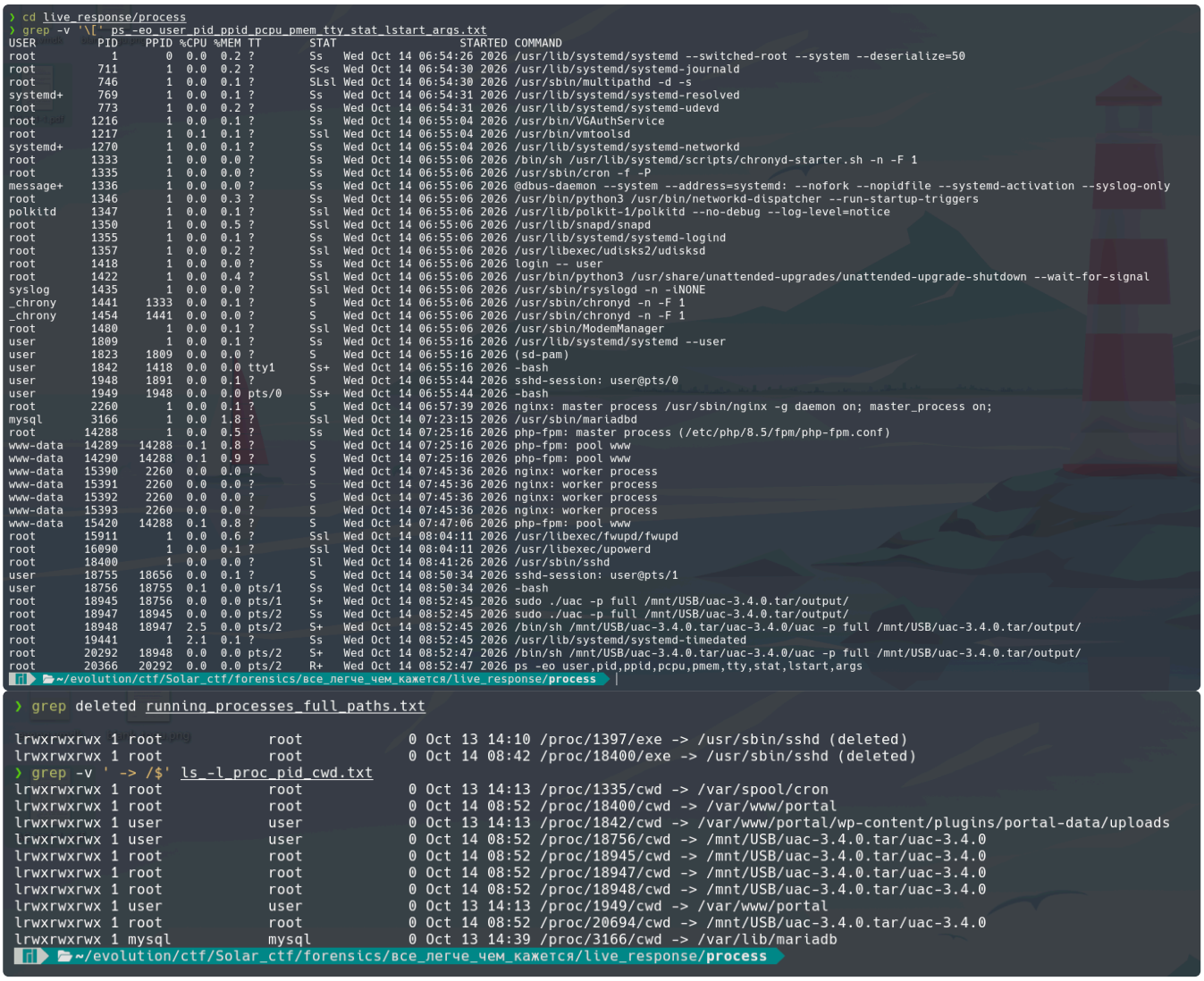
1. Ищем подозрительный процесс

После распаковки переходим в `live_response/process/`. Здесь лежат результаты `ps`, информация из `/proc` и восстановленные исполняемые файлы.

```
cd live_response/process

grep -v '\[' ps_-eo_user_pid_ppid_pcpu_pmem_tty_stat_lstart_args.txt
grep 'deleted' running_processes_full_paths.txt
grep -v ' -> /$' ls_-l_proc_pid_cwd.txt
```

Зачем три команды? Первая убирает часть шума от потоков ядра с помощью флага *-v* , вторая показывает процессы с удалённым с диска `exe`, а третья процессы, у которых рабочая директория не `/`.



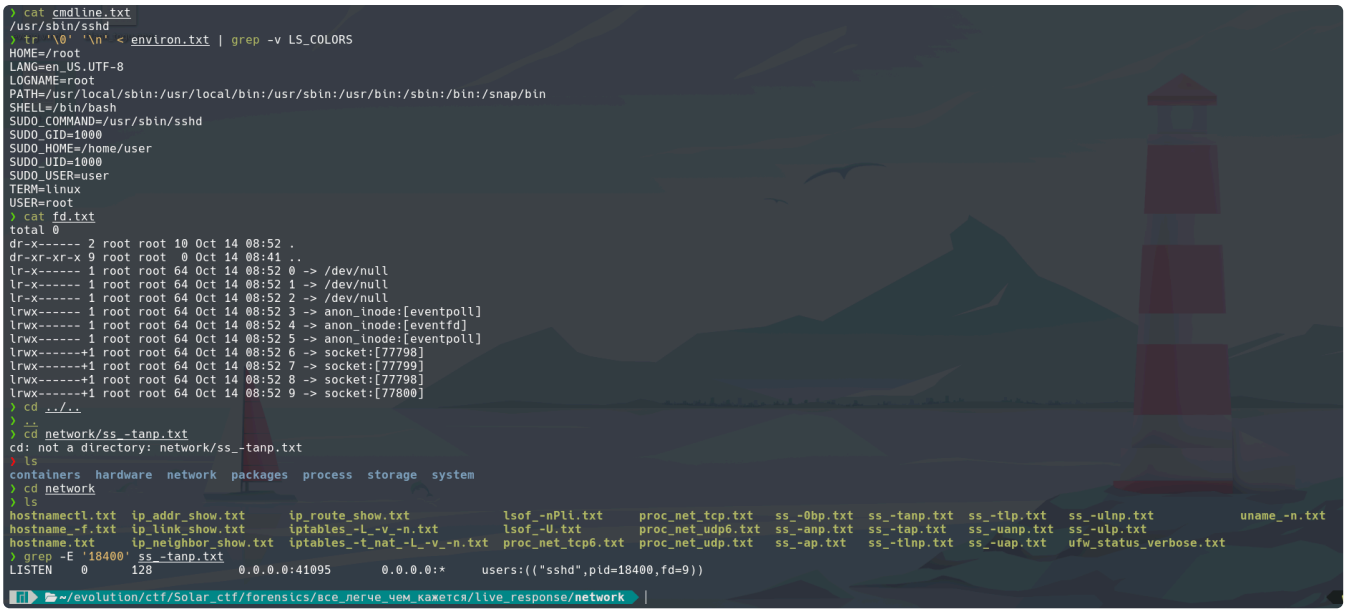
И тут нам должно бросаются в глаза три записи:

```
/proc/1397/exe  -> /usr/sbin/sshd (deleted)
/proc/18400/exe -> /usr/sbin/sshd (deleted)
/proc/18400/cwd -> /var/www/portal
```

Почему это интересно? `sshd (deleted)` `сам по себе ещё не вредонос`: так бывает после обновления пакета. Но у `PID 18400` рабочая директория — веб-корень `/var/www/portal`, а время старта отличается от обычного системного `sshd`. Уже повод проверить его отдельно.

2. Что скрывается за PID 18400?

```
cat proc/18400/cmdline.txt
tr '\0' '\n' < proc/18400/environ.txt | grep -E 'SUDO_(COMMAND|USER)'
grep '18400' ../network/ss_-tanp.txt
```



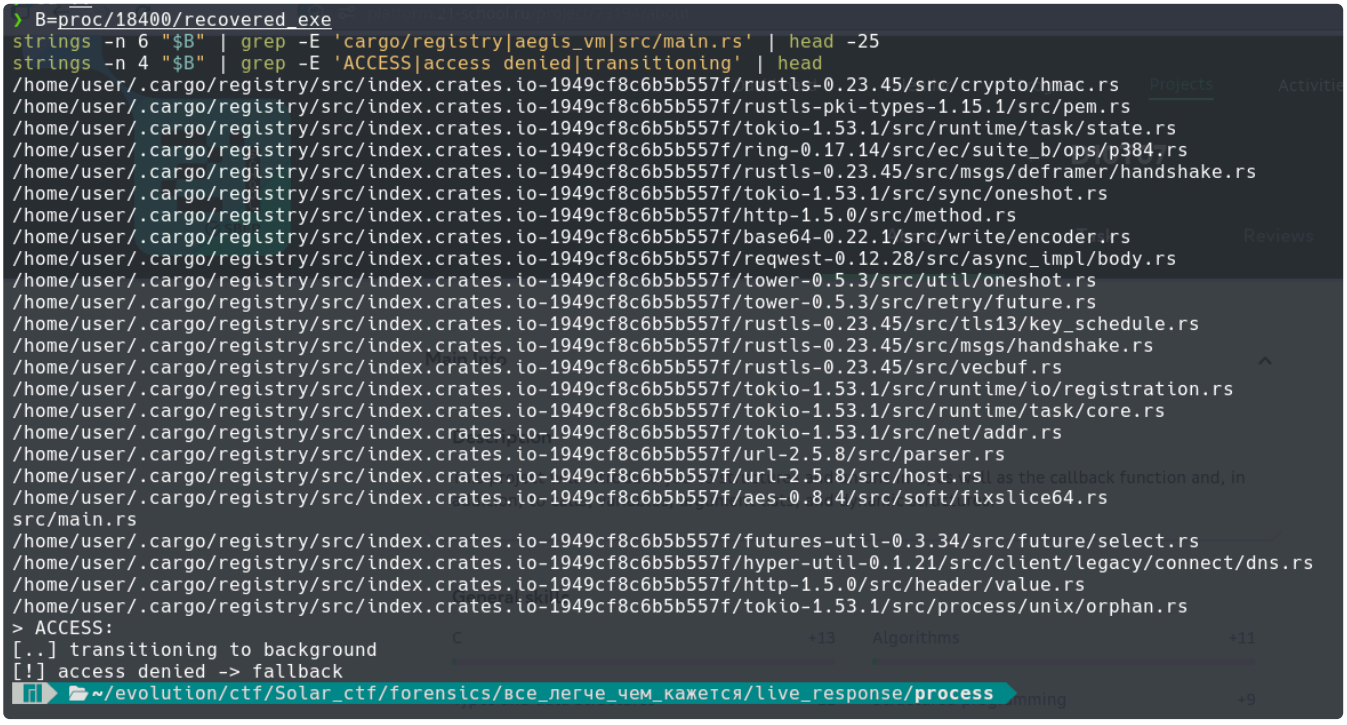
В окружении видим `SUDO_COMMAND=/usr/sbin/sshd` и `SUDO_USER=user`, а в сети — `0.0.0.0:41095 LISTEN`. То есть процесс с именем `sshd` запускали через `sudo`, и он слушает необычный порт. Похоже на маскировку — давайте проверим сам файл.

3. Заглядываем внутрь бинаря

Сначала самый дешёвый способ `strings`.

```
B=proc/18400/recovered_exe
strings -n 6 "$B" | grep -E 'cargo/registry|aegis_vm|src/main.rs' | head -25
strings -n 4 "$B" | grep -E 'ACCESS|access denied|transitioning' | head
```

Что видим? Бинарь написан на Rust. Среди строк встречается `aegis_vm` след обфускации RustAegis. А `ACCESS` и `access denied` намекают, что внутри есть проверка ввода.



И ничего интересного не находим :(

4. Находим проверку пароля

Чтобы не разбирать весь Rust-бинарь, цепляемся за текст `> ACCESS:`. Сначала находим смещения строк:

```
grep -aboE 'ACCESS|access denied|transitioning' "$B"
readelf -lW "$B" | grep LOAD
```

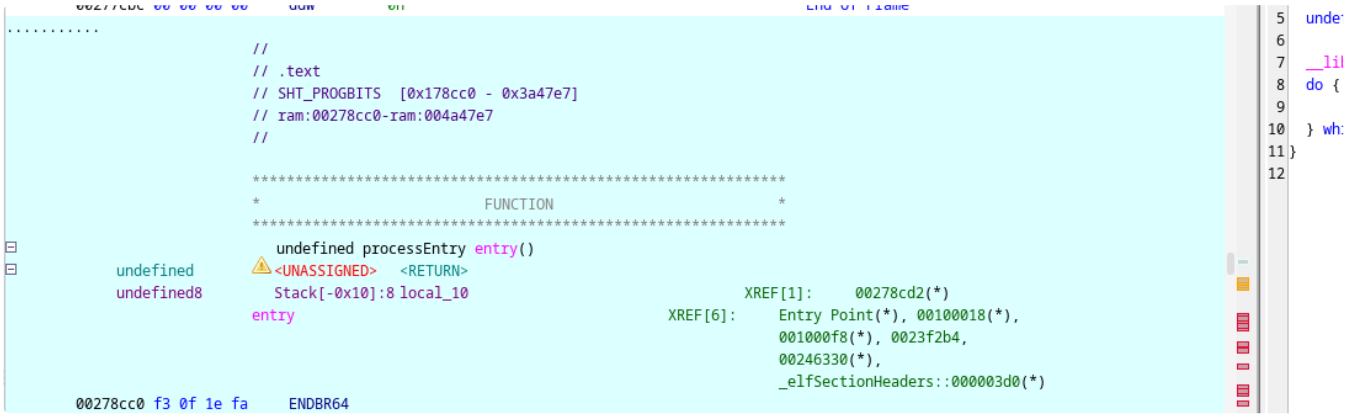
Дальше открываем **`recovered_exe` в Ghidra** находим строку `ACCESS`, переходим к *References / Xrefs* и смотрим вызывающую функцию.

По разбору получились такие адреса:

```
0x19e843  →  "> ACCESS:"
0x19e8a4  →  "access denied"
0x19f1dc  →  "transitioning to background"
```

И тут самое интересное: вместо обычных `if/else` control-flow flattening, то есть логика разложена по состояниям и таблице переходов. Но нам достаточно понять два места:

```
state 2 → собирает ожидаемый пароль и сравнивает с вводом
state 3 → собирает и выводит секрет при верном пароле
```



5. Как достали флаг

Ожидаемый пароль и секрет собираются из четырёх зашифрованных фрагментов каждый. Например, первый фрагмент секрета расшифровывается в `SOLARCTF`:

```
data = open('proc/18400/recovered_exe', 'rb').read()
d, key = data[0x38980:0x38988], data[0x389f0:0x389f8]
out = bytearray(8)
k = 0x5e
```

```
for i in range(8):
    out[7-i] = k ^ key[i] ^ d[i]
    k = (d[i] + k) & 0xff
print(out)  # bytearray(b'SOLARCTF')
```

Остальные функции устроены по разному вручную переписывать каждую сомнительное удовольствие. Поэтому в решении использовался *Unicorn* он эмулирует отдельные функции расшифровки из ELF, а не запускает вредонос как обычную программу.

```
Пароль:  ergePAsdf^#$&!SSUAH*S
```

И так получаем флаг: `SOLARCTF{NGC_n3v3r_sl33ps_0nl1n3}`